NVIDIA

Nvidia is one of those companies everyone talks about now. But it wasn't always like that, not even close. It started back in 1993 with Jensen Huang, Chris Malachowsky, and Curtis Priem, and for a long time it was basically just a gaming company GPUs, graphics cards, that whole space. Later it moved into things like SoCs and APIs, spreading into data science, AI, mobile, even automotive. Still, if you told someone in the early 2000s that this company would be worth trillions because of AI, they probably wouldn't have taken you seriously.

Somewhere in the early-mid 2000s though, Nvidia made a pretty bold move. They invested over a billion dollars into CUDA, which is a lot, especially back then. CUDA is basically a software platform that lets GPUs handle tons of calculations in parallel, not just graphics. It connects hardware to software in a way that gives developers more direct control over the GPU, CPU, and tools, all of it. At the time it must've felt like a gamble, maybe even a stretch, since Nvidia wasn't huge yet and the returns weren't immediate. But looking back now, it's hard not to think this is where things really started shifting.

For years, Nvidia was mostly known for GeForce cards. Gamers loved them, but outside that, not much attention. Then around 2012, deep learning started picking up, and researchers noticed something interesting GPUs, originally meant for rendering graphics, were actually really good at the kind of math neural networks need. Kind of unexpected. Nvidia leaned into that pretty early and shifted focus, and that's honestly where things start getting interesting.

The Problem

So what were they actually trying to solve? At first glance it sounds simple, just make better chips. But it's not really that. GPUs work differently from CPUs instead of doing one thing at a time, they handle thousands of operations simultaneously, which turns out to be perfect for AI, simulations, and heavy computation. Nvidia saw that early, maybe earlier than most. The real challenge, though, was turning a gaming-focused chip into something useful across completely different industries, and that's not trivial.

Honestly, the harder part might not have even been hardware. Hardware companies exist everywhere, and fast chips aren't some mysteries. The real issue was accessibility. What's the point of a powerful chip if developers can't easily use it? If writing software for it is painful, adoption just stalls. So Nvidia had to think beyond hardware, which is a bit unusual for a hardware company if you think about it.

There's another angle too. Once Nvidia became the go-to for AI chips, it also became a target. Competitors had every reason to step in, some to beat them on price, others just to avoid depending on a single company. Even governments got involved, since AI chips started looking less like products and more like strategic assets. So the problem kind of evolved not just "get ahead," but "stay ahead while everyone's trying to catch up," which is a different kind of challenge.

The Strategy

Before getting into Nvidia's approach, it helps to look at what others were doing, because in a way, Nvidia's strategy makes more sense in that context. A bunch of companies jumped in once AI hardware became obviously important — AMD, Intel, Google, Amazon, Microsoft all took their shot.

AMD pushed its Instinct GPUs with solid specs and often cheaper pricing. Intel tried re-entering GPUs and also built AI accelerators, probably betting on scale and manufacturing. Google built TPUs mainly for its own use. Amazon and Microsoft followed a similar path with their own chips Triennium, Inferential, Maia. The idea was pretty straightforward: build competitive hardware, price it well, and win on performance-per-dollar. Honestly, that logic isn't wrong. In most hardware markets, it works.

But Nvidia didn't really play that same game, or at least not only that game. Instead of just competing on specs, they kind of shifted the battlefield. If it was purely about faster chips, maybe they would've lost ground eventually, since that's how hardware usually goes someone builds something better. But CUDA changed things.

Because developers had already spent years building on CUDA, writing code, creating tools, learning the system, switching wasn't simple. It wasn't just swapping hardware, it meant redoing a lot of work, maybe too much work. That

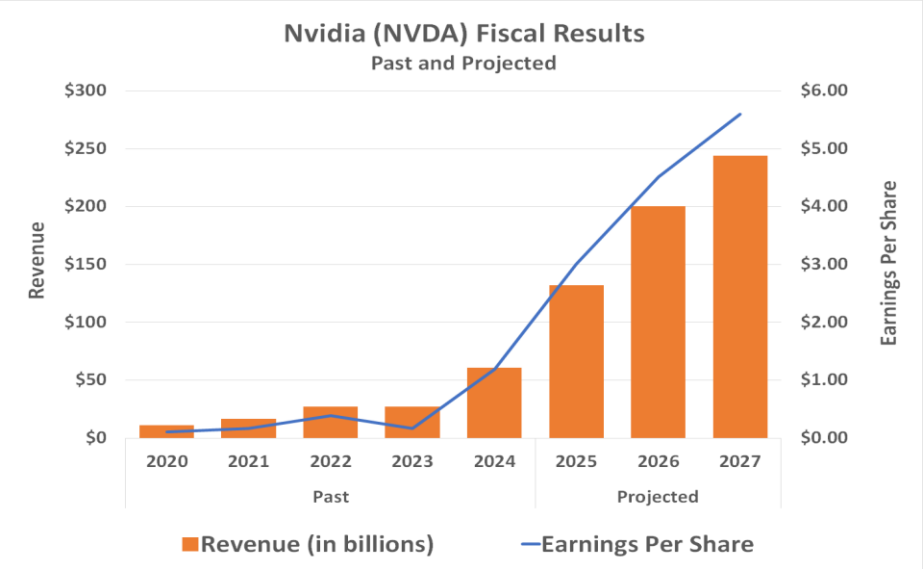
friction matters. I might be wrong here, but this feels like the core advantage not just better chips, but higher switching costs.

It's almost like a coordination problem. Developers stick with what others are using. Even if something else is technically just as good, the ecosystem matters tutorials, community, existing code, all of that keeps people where they are. You see this in other industries too, platforms, not just products.

And Nvidia kept reinforcing that. New architectures every couple of years, Pascal, Ampere, Hopper, always moving forward. So even if someone wanted to switch, they'd constantly be catching up. Not easy.

A few things kind of stand out here:

- Margins stayed high even with competition, which isn't typical.
- The ecosystem became a real moat, hard to break.
- Big players like Google and Amazon mostly kept their chips internal.
- It stopped being a one-time sale and became more like an ongoing relationship.



Implementation

Execution mattered a lot to, it wasn't just the idea. Nvidia pushed CUDA into universities early, offering cheap or free access, so students learned on it by default. That compounds over time. They also built specialized libraries, CUDNN (CUDA Deep Neural Network library) for example, which made developers' lives easier.

At some point, they shifted serious focus toward data centres and AI, even when gaming was still the main revenue source. That takes conviction, maybe even risk tolerance. They also expanded beyond just chips, acquiring Mellanox and moving into networking, building out more of the stack. And they kept releasing new hardware consistently, a rhythm that probably helped keep customers locked in.

Those students who learned CUDA eventually became engineers, and when they entered the industry, they stuck with what they knew. Makes sense, learning something new takes effort. Over time, all of this compounded.

The Result

The results were pretty dramatic, honestly. When the AI boom hit, Nvidia was already in position. Most companies needed their chips, not because there were zero alternatives, but because nothing matched the full ecosystem. That drove massive growth in data-centre revenue, and eventually pushed Nvidia into that trillion-dollar range.

Competitors didn't exactly fail. AMD has strong hardware, and Google's TPUs are widely used internally. But matching ecosystem depth, that's been harder. From what I've seen, that's really the gap.

If I had to sum it up, and this might be oversimplifying a bit, Nvidia didn't win just by making faster chips, and others didn't lose just because they were slower. Nvidia controlled the layer between hardware and users, and made that layer indispensable. That's a pretty powerful position to be in, probably more durable than just being "the fastest" at any point in time.

Reference/citation :

Google: https://share.google/7aVBFXCXJaTWV5VdW

Wikipedia: https://en.wikipedia.org/wiki/Nvidia

Scribd: https://www.scribd.com/document/350483951/Nvidia

Chat Gpt:

What is CUDA and why was it important for Nvidia?

What all problems does nvidia face because of it competitors?

Compare Nvidia’s approach with its competitors?

Explain Nvidia’s strategy used to overcome its competitors?

What role did long-term thinking play in Nvidia’s growth?